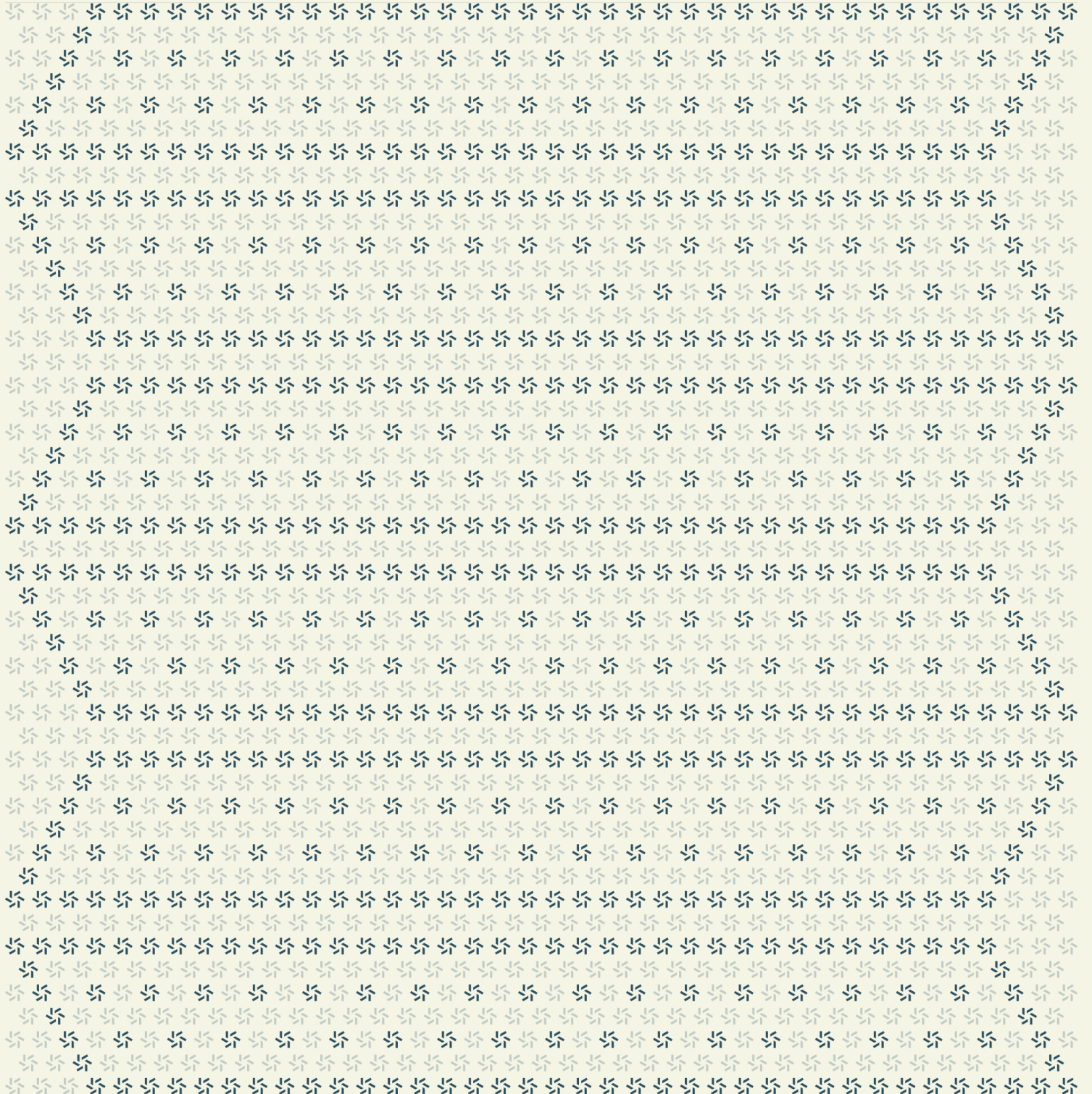


September 24, 2026

Token 2022 Changes

Solana Application Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About Token 2022 Changes	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	11
2.5. Project Timeline	11
3. Discussion	12
3.1. Rounding behavior for scaled UI amounts	12
4. System Design	13
4.1. Component: Scaled UI Amount extension	13
4.2. Component: Extension account length helper	13
4.3. Component: Memo Transfer extension	13
4.4. Component: CPI Guard for WithdrawExcessLamports	14

4.5.	Component: TLV Account Resolution	14
------	-----------------------------------	----

5.	Assessment Results	15
----	--------------------	----

5.1.	Disclaimer	15
------	------------	----

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Anza Technology, Inc. from September 16th to September 18th, 2026. During this engagement, Zellic reviewed Token 2022 Changes's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Does scaled UI amount conversion follow the intended truncation policy without overstating displayed balances?
 - Do the account-length helpers derive the required extensions and calculate account size without intermediate heap allocation on chain?
 - Does Memo Transfer accept only the intended memo programs, and does an enabled CPI Guard block `WithdrawExcessLamports` through CPI?
 - Does TLV account resolution preserve account order and prevent signer or writable privilege escalation without quadratic heap allocation?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Token 2022 Changes programs, there were no security vulnerabilities discovered.

Zellic recorded its notes and observations from the assessment for the benefit of Anza Technology, Inc. in the Discussion section ([3.7](#)).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	0
Informational	0

2. Introduction

2.1. About Token 2022 Changes

Anza Technology, Inc. contributed the following description of Token 2022 Changes:

Token-2022 is a Solana program for fungible and non-fungible tokens. It provides the interface and implementation that third parties use to create and manage tokens. This update optimizes memory usage for transfers involving a transfer-hook, and supports the memo v4 program in the memo-transfer extension.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the programs.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect

its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped programs itself. These observations — found in the Discussion ([3.7](#)) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Token 2022 Changes Programs

Type	Rust
Platform	SVM-compatible
Target	Scaled UI Amount extension (PR #1340)
Repository	https://github.com/solana-program/token-2022 ↗
Version	Diff from Git commit d6e57def6a06154c4edb6cb8f3ab27494b89d6d8 to Git commit e983a9678e5b72546ef2a16e77523d19d57cb277 in branch plutohan/issue-1324-scaled-ui-truncation
Programs	interface/src/extension/scaled_ui_amount/mod.rs
Target	Extension account-length helper (PR #1231)
Repository	https://github.com/solana-program/token-2022 ↗
Version	Diff from Git commit 71b4fe8e515390b986c897b2640548e183619c8d to Git commit e752f1871049352f6c7896aa0280dd46928bb68d in branch account-len-helpers
Programs	interface/src/extension/account_len.rs interface/src/extension/mod.rs program/src/processor.rs clients/rust-legacy/src/token.rs

Target	Memo Transfer extension (PR #1212)
Repository	https://github.com/solana-program/token-2022 ↗
Version	Diff from Git commit a0f5193635cf47e71ba4db2ef547a048d401d5e0 to Git commit 071043f8c9a7474f681a47a5511d23f713de6f81 in branch febo/use-memo-v4
Programs	program/src/extension/memo_transfer/mod.rs clients/rust-legacy/src/token.rs
Target	CPI Guard WithdrawExcessLamports (PR #1196)
Repository	https://github.com/solana-program/token-2022 ↗
Version	Diff from Git commit 6c2a262c688d4f20cc16fc55f00fa31e6118b7e8 to Git commit d15dc486d88b3b80a476b9f9470a05e95fe9a3a2 in branch bit2swaz:fix/withdraw-excess-lamports-cpi-guard-clean
Programs	program/src/processor.rs
Target	TLV Account Resolution (PR #199)
Repository	https://github.com/solana-program/libraries ↗
Version	Diff from Git commit 9850ca1a88c9c0ebdc5730fb9414bd54edb718c2 to Git commit e46f0968731ba1ab84c5fa8709619035289c297a in branch stranzhay:fix/tlv-cpi-resolution-allocations
Programs	tlv-account-resolution/src/state.rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 0.8 person-weeks. The assessment was conducted by two consultants over the course of 0.6 calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✈ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Ziling Chen
✈ Security Engineer
ziling@zellic.io ↗

Xuehao Guo
✈ Security Engineer
guo@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

September 16, 2025 Start of primary review period

September 18, 2025 End of primary review period

3. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

3.1. Rounding behavior for scaled UI amounts

The `ScaledUiAmountConfig::amount_to_ui_amount` function converts a raw token balance into its scaled UI representation using the active multiplier and the mint's decimal precision. [PR #159](#) [↗] deliberately changed this conversion from rounding to the nearest value to truncation toward zero, as rounding can display a greater amount than the exact scaled balance. For example, a raw amount of 101 with a multiplier of 0.99 and two decimals has an exact scaled value of 0.9999. Rounding to two decimal places displays 1; truncation displays 0.99.

[PR #1340](#) [↗] addressed a separate edge case caused by truncating before applying the mint's decimal scaling. With a raw amount of 1, a multiplier of 0.5, and two decimals, the mathematically scaled value is 0.005. The previous implementation truncated the intermediate value to zero and displayed 0. By formatting the floating-point scaled value directly to two decimal places, [PR #1340](#) instead displayed 0.01. This kept small nonzero balances visible but restored rounding to the nearest value throughout the conversion from raw amounts to UI amounts, reversing the deliberate behavior introduced in [PR #159](#).

We raised this discrepancy with the Anza Technology, Inc. team during the assessment. The team confirmed that rounding to the nearest value was not intended and merged [PR #1480](#) [↗], an exact revert of [PR #1340](#). Truncation therefore remains the intended behavior. The conversion follows the policy established in [PR #159](#) instead of rounding at the displayed precision, so a nonzero value below that precision may appear as zero. [PR #1481](#) [↗] separately reverted the equivalent behavior in the JavaScript clients.

4. System Design

This section summarizes the notable implementation changes in the five pull request diffs reviewed during this assessment. As the scope comprised of isolated changes rather than an entire program, these notes focus on the behavior and data flow introduced or modified by each pull request.

4.1. Component: Scaled UI Amount extension

These are the notable changes in the Scaled UI Amount extension.

- In [PR #1340](#), `ScaledUiAmountConfig::amount_to_ui_amount` multiplies the raw amount by `total_multiplier` without truncating first then formats the product at the mint's decimal precision. The conversion now rounds to the nearest value at a fixed precision instead of truncating toward zero.
- This makes some small positive values visible at the displayed precision and may also produce a displayed amount above the exact scaled balance. For example, one raw unit with a 0.5 multiplier and two decimals is displayed as 0.01 instead of 0; with 101 raw units and a 0.99 multiplier, the output is 1 instead of 0.99.

4.2. Component: Extension account length helper

These are the notable changes in the extension account length helper.

- [PR #1231](#) introduces `try_for_each_required_init_account_extension`, which walks the extension entries in the supplied mint TLV data in TLV order. The helper invokes a callback for each account extension required during initialization instead of building an intermediate collection. For example, `TransferFeeConfig` emits `TransferFeeAmount`; `NonTransferable` emits `NonTransferableAccount` followed by `ImmutableOwner`.
- The new `try_calculate_account_len_from_mint_data` helper validates the mint representation and rejects explicitly requested extensions that are not account extensions. It combines the required and requested extensions, removes duplicates, and then applies the padding adjustment used when an account containing extensions would otherwise have the same length as a multi-sig account.
- The `InitializeAccount` and `GetAccountDataSize` processor paths now derive and size the extensions required by the mint without constructing an intermediate `Vec<ExtensionType>`. The Rust legacy client also uses the traversal helper but collects the emitted extensions into a `Vec` when constructing instructions outside the program. Checks for mint ownership and malformed TLV data remain in place. A u64 bitset deduplicates extension types; the pull request adds a test confirming that every current `ExtensionType` maps to a distinct bit below 64.

4.3. Component: Memo Transfer extension

These are the notable changes in the Memo Transfer extension.

- In [PR #1212](#), the program adds Memo v4 to its memo program allowlist. Memo v1 and v3 remain accepted for backward compatibility. For transfers into accounts that require a memo, the most recently processed sibling instruction in the current invocation context must use one of these three program IDs. If no such sibling exists or its program ID is not allowlisted, the transfer fails with `TokenError::NoMemo`.
- The Rust legacy client now uses the Memo v4 program by default when `TokenMemo::to_instruction` builds an attached memo. Only the selected memo program changes; the memo bytes and configured signer addresses stay the same.

4.4. Component: CPI Guard for `WithdrawExcessLamports`

This is a notable change in the CPI Guard for `WithdrawExcessLamports`.

- In [PR #1196](#), `process_withdraw_excess_lamports` now reads a source token account's `CpiGuard` extension, if present, after the existing code confirms that Token-2022 owns the source account, rejects native token accounts, and validates the withdrawal authority against the token account's stored owner. If `lock_cpi` is enabled and Token-2022 is executing through CPI, the function returns `TokenError::CpiGuardTransferBlocked`.

4.5. Component: TLV Account Resolution

These are the notable changes in the TLV Account Resolution.

- In [PR #199](#), `check_account_infos` and `add_to_cpi_instruction` now build a lookup of account keys and borrowed data once per call rather than once per extra account meta. Cumulative heap allocation grows linearly with the length of the list instead of quadratically. This behavior is relevant to SBF because its bump allocator does not reclaim memory during execution.
- CPI resolution adds each account to the lookup immediately after resolving it, allowing later metas to reference that account. Account order, rejection of missing accounts, and deescalation of signer and writable privileges are retained.
- Resolved account infos are collected separately because cached data borrows remain live during the resolution loop. The code appends them after releasing those borrows. Both resolution paths also return immediately for an empty list of extra account metas to avoid unnecessary account conversion and data borrowing.

5. Assessment Results

During our assessment on the scoped Token 2022 Changes programs, there were no security vulnerabilities discovered.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.